Faculty of Engineering and Technology

Department of Electrical and Computer Engineering

ENCS3390: Operating System Concepts

Project 1 Report

Prepared by: Lara Saleh Abu Asfour

ID: 1221484

Instructor: Abdel Salam Sayyad

Section: 4

Date: 8/12/2024

Abstract

This project looks at the "enwik8" dataset to find the top 10 most frequent words using three different methods: a basic sequential approach, a multiprocessing approach, and a multithreading approach. We compare how each method performs by measuring how long they take with different numbers of processes and threads. The results are then analyzed to see how well each method scales, based on the number of CPU cores available. The goal is to understand how parallel processing can speed up analyzing large text files.

Table of Contents

Abstract II

Table of Figures IV

List of Tables V

1. Theory 1

1.1 Naïve Approach 1

1.2 Multiprocessing Approach 1

1.3 Multithreading Approach 1

2. The Environment Setup 2

3. Naïve, Multiprocessing and Multithreading requirements 2

3.1 Naïve Approach Code 2

3.2 Multiprocessing Approach Code 4

3.3 Multithreading Approach Code 7

4. Amdahl's law 10

5. The Comparison between the performance of 3 approaches 11

Conclusion: 12

References: 13

Table of Figures

Figure 1: The output of Naive approach code. 3
Figure 2: The output of multiprocessing code (2 children). 5
Figure 3: The output of multiprocessing code (4 children). 5
Figure 4: The output of multiprocessing code (6 children). 5
Figure 5: The output of multiprocessing code (8 children). 6
Figure 6: The output of multithreading code (2 threads). 7
Figure 7: The output of multithreading code (4 threads). 8
Figure 8: The output of multithreading code (6 threads). 8
Figure 9: The output of multithreading code (8 threads). 8

List of Tables

Table 1: The Execution time and the Performance of Multiprocessing. 6
Table 2: The Execution time and the Performance of Multithreading. 9
Table 3: The Execution time and Performance for each approach. 11

1. Theory

1.1 Naïve Approach

The naive approach is the simplest way to solve the problem, where everything is done step by step in a single thread. It processes the dataset one piece at a time, without using any parallelism or advanced techniques. While it's easy to write and understand, this approach is slow and doesn't make full use of modern multi-core processors. The more data there is, the longer it takes, making it unsuitable for large datasets. However, if the dataset is small or if simplicity is the priority, the naive approach can get the job done.

1.2 Multiprocessing Approach

The multiprocessing approach speeds things up by dividing the work among multiple independent processes, each running on a separate core. Each process works on its part of the dataset without interfering with the others, which helps avoid data corruption. To share results, processes use tools like shared memory or messaging, which can add some complexity. This method works great for large datasets because it takes full advantage of a multi-core processor, but creating and managing multiple processes can slow things down if the overhead gets too high. Multiprocessing is ideal when the tasks can be split up and don't need constant communication between parts.

1.3 Multithreading Approach

The multithreading approach uses multiple threads within a single process to work on tasks at the same time. Since all threads share the same memory, they can communicate quickly, but this also means you need to carefully manage access to avoid issues like race conditions. Threads are lightweight compared to processes, so they're faster to create and switch between, making this approach efficient for tasks that need frequent sharing of data. However, if something goes wrong with one thread, it can affect the whole program, so you need to handle them carefully. This method is a great choice when you want to balance performance and resource efficiency, especially for tasks with lots of shared data.

2. The Environment Setup

- Computer Specs:
  Cores: 4
  Speed: 2.80 GHz
  Memory: 16.0 GB
- OS: Microsoft Windows 11 Pro and I use Ubuntu and Linux to do this project
- Programming language: C on Linux
- IDE tool: Code::Blocks on Ubuntu
- I did not use a virtual box I used Ubuntu and I installed codeblocks on it.

3. Naïve, Multiprocessing and Multithreading requirements

3.1 Naïve Approach Code

This code reads a text file (text88.txt) and identifies the top 10 most frequent words using a naive approach. It processes the text word by word, converting each word to lowercase to ensure case-insensitive comparisons. Each word is either added to an array of structs (WordCount) with its frequency initialized to 1, or its frequency is incremented if the word already exists in the array. A linear search (findWordIndex) is used to check for existing words, and the qsort function sorts the array by word frequency in descending order. Finally, the program prints the 10 most frequent words along with their frequencies and measures the total execution time using gettimeofday.

► The naive approach code satisfies the requirements by:

- Sequentially processing the text using simple functions (fscanf, strcmp).
- Storing words and their frequencies in an array of structs (WordCount).
- Using linear searches (findWordIndex) to update or insert words.
- Sorting using qsort.
- Employing basic APIs (fopen, tolower, strcmp) to implement the logic.

► The output for naïve approach code:

```
Top 10 most frequent words using Naive approach:
the: 1061396
of: 593677
and: 416629
one: 411764
in: 372201
a: 325873
to: 316376
zero: 264975
nine: 250430
two: 192644
Execution time: 960.92 seconds
Parallel time: 960.87 seconds
Serial time: 0.05 seconds

Process returned 0 (0x0)   execution time : 960.923 s
Press ENTER to continue.
```

Figure 1: The output of Naive approach code.

► The naive approach is the slowest because it processes the entire file sequentially using a single thread, the Execution Time of it = 960.92 s and the Performance = 0.001041 (1/s)

3.2 Multiprocessing Approach Code

This program counts the frequency of words in a text file (text88.txt) using multiprocessing to divide the workload between multiple child processes. It employs shared memory for communication between processes to store the words and their frequencies. The file is divided into chunks, and each child process processes its assigned chunk. After all child processes finish, the parent process consolidates the results, sorts the word frequencies in descending order, and prints the top 10 most frequent words. The program also measures and displays the execution time.

► API and Functions Used in the Code:

- API: fopen, fseek, ftell, fscanf, fclose
  To open, navigate, and read chunks of the file.

- Function: toLowerCase
  Converts words to lowercase using tolower for case-insensitive comparisons.

- API: shm_open, mmap, ftruncate, munmap, shm_unlink
  Creates and manages shared memory for communication between processes.

- API: fork, wait
  Creates child processes and ensures the parent waits for their completion.

- Functions: findWordIndex
  Searches for existing words in the shared words array using strcmp.

- Functions: strcpy
  Copies new words to the words array.

- API: qsort
  Sort the words array by frequency in descending order using a custom comparison function.

- API: printf

  Measures execution time.

► The output of Multiprocessing Approach for (2, 4, 6, 8 children):

```
Top 10 most frequent words using multiprocessing (2 childs):
the: 1061377
of: 593668
and: 416621
one: 411757
in: 372200
a: 325869
to: 316373
zero: 264973
nine: 250429
two: 192644
Execution time: 545.26 seconds

Process returned 0 (0x0)   execution time : 545.268 s
Press ENTER to continue.
```

Figure 2: The output of multiprocessing code (2 children).

```
Top 10 most frequent words using multiprocessing (4 childs):
the: 1061330
of: 593668
and: 416620
one: 411753
in: 372190
a: 325865
to: 316374
zero: 264974
nine: 250427
two: 192642
Execution time: 493.56 seconds

Process returned 0 (0x0)   execution time : 493.567 s
Press ENTER to continue.
```

Figure 3: The output of multiprocessing code (4 children).

```
Top 10 most frequent words using multiprocessing (6 childs):
the: 1061320
of: 593655
and: 416614
one: 411743
in: 372189
a: 325867
to: 316373
zero: 264968
nine: 250424
two: 192640
Execution time: 406.92 seconds

Process returned 0 (0x0)   execution time : 406.928 s
Press ENTER to continue.
```

Figure 4: The output of multiprocessing code (6 children).

```
Top 10 most frequent words using multiprocessing (8 childs):
the: 1061294
of: 593639
and: 416620
one: 411751
in: 372191
a: 325867
to: 316373
zero: 264967
nine: 250426
two: 192636
Execution time: 416.75 seconds

Process returned 0 (0x0)   execution time : 416.754 s
Press ENTER to continue.
```

Figure 5: The output of multiprocessing code (8 children).

Table 1: The Execution time and the Performance of Multiprocessing.

	Execution Time (s)	Performance (1/s)
2 process	545.268	0.001834
4 process	493.56	0.002026
6 process	406.92	0.002457
8 process	416.75	0.0024

- So when we increase the number of children, then the Execution time will decrease but when we use 6 children then the execution time of it is less than the execution time of 8 children, this is because context switching or the core numbers.

3.3 Multithreading Approach Code

This program uses multithreading to count the frequency of words in a file and identifies the top 10 most frequent words. It divides the file into chunks, assigns each chunk to a thread, and uses a shared array to store word counts. The threads process their chunks independently but merge results into the shared array using a mutex for synchronization. Finally, the shared array is sorted, and the top 10 words are printed.

► API and Functions Used in the Code:

Functions:

- fseek, ftell, fopen, fscanf, fclose: For file handling.
- malloc, realloc, free: For memory management.
- strcmp, strcpy: For string comparison and copying.
- qsort: For sorting the array by frequency.

Threading APIs:

- pthread_create: To create threads.
- pthread_join: To wait for threads to finish.
- pthread_mutex_init, pthread_mutex_lock, pthread_mutex_unlock, pthread_mutex_destroy: For managing synchronization.

► The output of Multithreading Approach for (2, 4, 6, 8 threads)

```
Top 10 most frequent words using nultithreading (2):
the: 1061396
of: 593677
and: 416629
one: 411764
in: 372201
a: 325873
to: 316376
zero: 264975
nine: 250430
two: 192644
Execution time: 528.425035 seconds

Process returned 0 (0x0)   execution time : 528.427 s
Press ENTER to continue.
```

Figure 6: The output of multithreading code (2 threads).

```
Top 10 most frequent words using nultithreading (4):
the: 1061396
of: 593677
and: 416629
one: 411764
in: 372201
a: 325873
to: 316376
zero: 264975
nine: 250430
two: 192644
Execution time: 448.516954 seconds

Process returned 0 (0x0)   execution time : 448.519 s
Press ENTER to continue.
```

Figure 7: The output of multithreading code (4 threads).

```
Top 10 most frequent words using nultithreading (6):
the: 1061396
of: 593677
and: 416629
one: 411764
in: 372201
a: 325874
to: 316376
zero: 264975
nine: 250430
two: 192644
Execution time: 434.187205 seconds

Process returned 0 (0x0)   execution time : 434.189 s
Press ENTER to continue.
```

Figure 8: The output of multithreading code (6 threads).

```
Top 10 most frequent words using nultithreading (8):
the: 1061396
of: 593677
and: 416629
one: 411764
in: 372201
a: 325874
to: 316376
zero: 264975
nine: 250430
two: 192644
Execution time: 407.135720 seconds

Process returned 0 (0x0)   execution time : 407.137 s
Press ENTER to continue.
```

Figure 9: The output of multithreading code (8 threads).

Table 2: The Execution time and the Performance of Multithreading.

	Execution Time (s)	Performance (1/s)
2 threads	528.425	0.001892
4 threads	448.516	0.00223
6 threads	434.187	0.002303
8 threads	407.135	0.002456

- As shown in the Figures when we increase the number of the threads, then the Execution time will decrease, it is the fastest performance and efficient memory usage, but requires careful synchronization to avoid race conditions, The use of mutexes ensures thread safety during updates to the shared array. However, contention for the mutex lock and thread synchronization might limit scalability as the number of threads increases.

4. Amdahl's law

Amdahl's Law provides a way to estimate the maximum theoretical speedup of a program when using multiple processors or cores, as Figure 1 shows, in the naïve approach code the Execution time is 960.923 seconds, the parallel time is 960.87 seconds and the serial time is 0.05 seconds.

$$S\ (\text{Serial Time Fraction}) = \frac{\text{Serial Time}}{\text{Total Exexution Time}} = \frac{0.05}{960.923} = 0.000052 * 100\% = 0.0052\%$$

$$P\ (\text{Parallel Time Fraction}) = \frac{\text{Parallel Time}}{\text{Total Exexution Time}} = \frac{960.87}{960.923} = 0.99995 * 100\% = 99.99\%$$

$$\text{Speedup (N = 2)} = \frac{1}{S + \frac{P}{N}} = \frac{1}{0.000052 + \frac{0.99995}{2}} = 1.9999$$

$$\text{Speedup(N = 4 number of cores on my laptop)} = \frac{1}{S + \frac{P}{N}} = \frac{1}{0.000052 + \frac{0.99995}{4}} = 4.00004$$

$$\text{Speedup (N = 6)} = \frac{1}{S + \frac{P}{N}} = \frac{1}{0.000052 + \frac{0.99995}{6}} = 5.997$$

$$\text{Speedup (N = 8)} = \frac{1}{S + \frac{P}{N}} = \frac{1}{0.000052 + \frac{0.99995}{8}} = 7.998$$

- Amdahl's Law helps us understand the limits of speeding up a program by adding more processors or cores. In this project, the naïve approach took 960.923 seconds to run, with nearly all the time (99.99%) spent on parallelizable tasks and only 0.05 seconds (0.0052%) on serial tasks. Using Amdahl's formula, the speedup was calculated for different core counts. With 2 cores, the speedup is nearly double at 1.9999. For 4 cores, which matches the laptop's capability, the speedup is 4.00004. Adding more cores further improves performance, reaching 5.997 with 6 cores and 7.998 with 8 cores. These results show that while adding cores improves speed, the small serial portion limits how much faster the program can get, which is exactly what Amdahl's Law predicts.

5. The Comparison between the performance of 3 approaches

This table has the Execution time for the Naive approach, execution time for different numbers of processes (2, 4, 6, 8), execution time for different numbers of threads (2, 4, 6, 8) and performance (calculated as 1 / Execution Time for each approach).

Table 3: The Execution time and Performance for each approach.

	Execution Time (s)	Performance (1/s)
Naïve approach	960.92	0.001041
2 threads	528.425	0.001892
4 threads	448.516	0.00223
6 threads	434.187	0.002303
8 threads	407.135	0.002456
2 process	545.268	0.001834
4 process	493.56	0.002026
6 process	406.92	0.002457
8 process	416.75	0.0024

The results clearly show how parallelism improves performance compared to the naïve approach. The naïve method, which runs sequentially, has the longest execution time at 960.92 seconds and the lowest performance (0.001041). With multithreading, execution time steadily decreases as more threads are added, from 528.425 seconds with 2 threads to 407.135 seconds with 8 threads, achieving the best performance at 0.002456. Similarly, multiprocessing reduces execution time, starting at 545.268 seconds with 2 processes and reaching its best performance of 0.002457 at 6 processes, though performance slightly drops at 8 processes due to overhead. Overall, both approaches are much faster than the naïve method and they have almost similar values, with multithreading being slightly more efficient because of its lower overhead

Conclusion:

This project highlighted the significant impact of parallelism on improving program performance, especially when handling large datasets like enwik8. While the naive approach was simple, its sequential nature made it the slowest and least efficient. Multiprocessing and multithreading, on the other hand, demonstrated the power of leveraging multi-core processors, with multiprocessing slightly edging out in performance due to its independent process model. However, multithreading offered a simpler implementation and better memory efficiency, making it a great alternative. Overall, the project underscored the importance of choosing the right approach based on the task requirements, balancing simplicity, resource usage, and execution speed.

References:

The link of the input file text8.txt - Google Drive